



Sede Regional Brunca

Campus Pérez Zeledón

Curso: Estructuras de Datos

Proyecto 1: Neon Tetris

Estudiante:

Luis Andres Elizondo Hernandez

Profesores: Saray María Castro Mora

Pablo Andrés Venegas Elizondo

II Ciclo - NRC 51005 - Grupo 82

27 de septiembre del 2026

Introducción y Descripción del Sistema

En este primer proyecto del curso de Estructuras de Datos desarrollamos una versión funcional de Tetris llamada "Neon Tetris" en C++, utilizando la biblioteca gráfica Raylib y el entorno Zinjal. El objetivo principal fue programar todas las estructuras de datos lineales vistas en clase (colas, pilas, listas enlazadas y ordenamientos) desde cero con nodos y punteros en memoria dinámica, sin usar las librerías estándar de C++ (STL).

El juego se desarrolla en un tablero de 10 columnas por 20 filas donde caen 7 tipos de piezas clásicas. Cada mecánica clave del juego fue pensada para resolverse con la estructura lineal que le corresponde de forma natural: la secuencia de piezas que van a salir usa una cola; la reserva de una pieza para después (hold) usa una pila; el historial para deshacer jugadas y el replay usa una lista doblemente enlazada; los eventos con temporizador usan una cola de prioridad; y el tablero completo se modela como una lista enlazada de filas. Además, implementamos dos métodos de ordenamiento (Burbuja y Merge Sort) para ordenar la tabla de puntajes guardada en JSON y comparamos sus tiempos de ejecución reales para ver cómo rinden en la práctica.

Arquitectura del Sistema y Relación entre Estructuras

El proyecto está dividido en tres carpetas para mantener el código ordenado: estructuras (clases de datos y punteros), lógica (piezas, rotaciones precalculadas y ordenamientos) y ui (pantallas y diseño visual con Raylib). En el archivo principal main.cpp se conectan todas las estructuras en un bucle que funciona de la siguiente manera:

- **Cola de piezas siguientes (Cola):** Se encarga de abastecer las piezas del juego. Para evitar que salgan piezas repetidas muchas veces seguidas o pase mucho tiempo sin una pieza larga, usamos el sistema oficial de bolsa de 7: se mezclan las 7 piezas clásicas con el algoritmo de Fisher-Yates en $O(n)$ y se encolan. Conforme caen, se van sacando con `desencolar()`, y en la pantalla se muestran las 3 siguientes consultando los primeros nodos con `obtenerEn(k)`. Al vaciarse la cola, se baraja y se encola una nueva bolsa automáticamente.
- **Pila para la pieza en espera (Pila):** La mecánica de Hold permite apartar la pieza actual para usarla después con la tecla Shift Derecho. Lo resolvimos con una pila de capacidad 1. Si la pila está vacía, la pieza actual se apila y se saca una nueva de la cola de piezas; si ya había una pieza guardada, se intercambian sacando la guardada y apilando la que estaba

cayendo. Se usa una bandera para que el jugador solo pueda hacer un cambio por turno hasta que la pieza caiga.

- **Tablero como lista de filas (Tablero):** En vez de ser una matriz estática en un bloque fijo de memoria, el tablero es una lista enlazada de 20 filas por 10 columnas donde cada celda apunta a sus vecinas (arriba, abajo, izquierda y derecha). Al fijarse una pieza, se revisa si hay filas llenas. Si las hay, se hace una animación visual de destello de 250 ms y luego limpiarLineas() desconecta los punteros verticales de las filas llenas, libera los nodos con delete y agrega filas vacías al inicio.

- **Lista doblemente enlazada para historial y replay (ListaDoble):** Cada vez que se coloca una pieza o se limpian líneas, se guarda una copia del tablero y del puntaje en un nodo de la lista doble. Al tener punteros siguiente y anterior, durante la partida el jugador puede presionar Z para deshacer jugadas (retroceder) o Y para rehacerlas (avanzar) en tiempo real. Cuando la partida termina en Game Over, esta misma lista permite el modo Replay, donde se puede recorrer toda la partida paso a paso o en reproducción continua automática cada 0.5 s.

- **Cola de prioridad para eventos programados (ColaPrioridad):** Planifica eventos dinámicos que cambian la partida: aumento de velocidad temporal, líneas de basura desde abajo (Terremoto) o bonos de puntaje. La cola mantiene los eventos ordenados por tiempo. Como el evento que debe ocurrir más pronto queda siempre al frente (en la posición 0), consultarlo con verTiempoFrente() toma $O(1)$, y al cumplirse el tiempo de la partida se extrae en $O(\log n)$ y se aplica su efecto.

- **Persistencia y ordenamiento de puntajes:** Al terminar una partida, el jugador puede ingresar su nombre y el puntaje se guarda en scores.json usando la clase ManejadorJSON. En la pantalla de Ranking, el jugador puede ver la lista de récords y puede alternar entre Bubble Sort ($O(n^2)$) y Merge Sort ($O(n \log n)$), viendo en pantalla el tiempo exacto que tardó cada uno en microsegundos.

Complejidad Teórica Asintótica (Notación Big-O)

En la siguiente tabla resumimos la complejidad teórica de las operaciones principales de las estructuras implementadas en el proyecto:

Estructura	Operación	Mejor	Promedio	Peor	Espacio	Descripción Breve
Cola FIFO	encolar()	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Inserta directo al final usando el puntero último.

Estructura	Operación	Mejor	Promedio	Peor	Espacio	Descripción Breve
Cola FIFO	desencolar()	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Saca del frente y libera el nodo con delete.
Cola FIFO	obtenerEn(k)	$O(1)$	$O(k)$	$O(n)$	$O(1)$	Recorre hasta la posición k para ver piezas futuras.
Pila LIFO	apilar() / desapilar()	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Inserta o extrae directamente en el tope.
Lista Doble	agregarEstado()	$O(1)$	$O(1)$	$O(1)$	$O(F \cdot C)$	Inserta al final; clona la matriz de 20x10 y puntaje.
Lista Doble	retroceder() [Undo]	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Paso al nodo anterior: actual = actual->anterior.
Lista Doble	avanzar() [Redo]	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Paso al nodo siguiente: actual = actual->siguiente.
Cola Prioridad	encolar()	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Inserta y se acomoda comparando con anteriores.
Cola Prioridad	desencolar()	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Saca evento del frente y reacomoda el arreglo.
Cola Prioridad	verTiempoFrente()	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Consulta directa al primer evento en arreglo[0].
Tablero	limpiarLineas()	$O(F \cdot C)$	$O(F \cdot C)$	$O(F \cdot C)$	$O(1)$	Reconecta punteros verticales y crea filas nuevas arriba.
Ordenamiento	Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Comparaciones e intercambios adyacentes de vecinos.
Ordenamiento	Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Divide recursivamente y mezcla sub-arreglos.

4. Comparación Empírica y Benchmark (Burbuja vs. Merge Sort)

Medimos con <chrono> en C++ el tiempo real en microsegundos ordenando listas aleatorias de jugadores con puntajes entre 0 y 100 000 para $N = 10, 100, 1\ 000$ y $10\ 000$ registros (promedio de 5 corridas por caso):

Tamaño (N)	Bubble Sort (µs)	Bubble Sort (ms)	Merge Sort (µs)	Merge Sort (ms)	Diferencia de Rendimiento
N = 10	0.90 µs	0.0009 ms	1.00 µs	0.0010 ms	Empate técnico (0.9x)
N = 100	117.50 µs	0.1175 ms	38.50 µs	0.0385 ms	Merge 3.05x más rápido
N = 1 000	12 333.00 µs	12.33 ms	650.00 µs	0.65 ms	Merge 18.97x más rápido
N = 10 000	1 309 150.00 µs	1 309.15 ms	5 857.00 µs	5.86 ms	Merge 223.51x más rápido

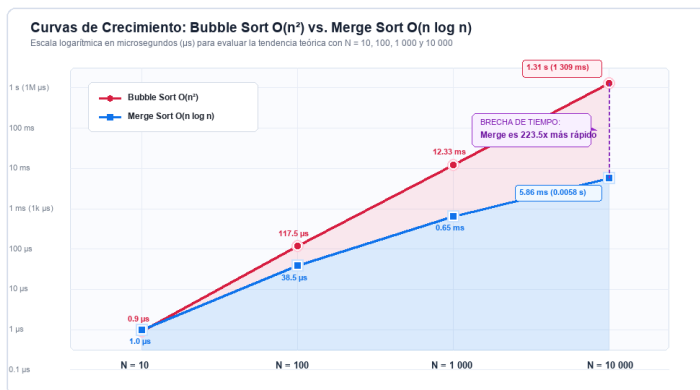


Figura 1: Curva de crecimiento en escala logarítmica comparando Bubble Sort y Merge Sort.

Análisis de resultados:

- Con $N = 10$, los dos algoritmos tardan casi lo mismo ($0.9 \mu s$ vs $1.0 \mu s$). Bubble Sort fue una fracción más rápido porque trabaja sobre el mismo arreglo sin llamadas a funciones recursivas ni reserva de memoria para la mezcla.
- Con $N = 100$, Merge Sort toma la delantera siendo 3 veces más veloz ($38.5 \mu s$ frente a $117.5 \mu s$).
- Con $N = 1\,000$, la diferencia se vuelve evidente: Merge Sort tarda solo 0.65 ms mientras que Bubble Sort tarda 12.33 ms (casi 19 veces más rápido).
- Con $N = 10\,000$, la diferencia es gigantesca: Bubble Sort tarda más de 1.3 segundos ($1\,309.15$ ms), lo que congelaría la pantalla, mientras que Merge Sort completa todo en menos de 6 milisegundos (5.86 ms), siendo más de 223 veces más rápido.
- En cuanto al crecimiento: al multiplicar los datos por 10 (de $1\,000$ a $10\,000$), Bubble Sort aumentó su tiempo unas 106 veces (de 12.33 ms a $1\,309.15$ ms). En cambio, Merge Sort solo aumentó 9 veces (de 0.65 ms a 5.86 ms), confirmando su comportamiento casi lineal $O(n \log n)$.

5. Respuestas a las preguntas de análisis

Pregunta 1: Comparación de $O(n^2)$ vs. $O(n \log n)$, umbral de diferencia y notación asintótica

Con pocos datos ($N = 10$), ambos algoritmos empatan técnicamente en rendimiento ($0.9 \mu s$ para Bubble Sort y $1.0 \mu s$ para Merge Sort). Para tamaños muy pequeños las pocas operaciones directas de Bubble Sort son tan rápidas que compensan no tener la complejidad de Merge Sort, el cual gasta tiempo extra llamando funciones recursivas y pidiendo memoria para su arreglo temporal. La diferencia comienza a notarse claramente a partir de $N = 100$, donde Merge Sort ya es 3 veces más rápido. Con $N = 1\,000$ ya es 19 veces más veloz, y con $N = 10\,000$ la brecha es total: Bubble Sort tarda 1.31 segundos y Merge Sort apenas 5.86 milisegundos (más de 223 veces más rápido).

Sí coincide plenamente con lo esperado: Bubble Sort es cuadrático $O(n^2)$ multiplicando su tiempo por 100 al multiplicar datos por 10 (en nuestra prueba aumentó 106 veces), mientras que Merge Sort escala como $O(n \log n)$ con un crecimiento mucho más suave y eficiente.

Pregunta 2: Representación del tablero como lista enlazada de filas y costo de operaciones

En un arreglo estático común de 20x10, cuando se llena una fila en el medio hay que hacer bucles para copiar y mover físicamente hacia abajo todas las celdas de las filas superiores, haciendo hasta 190 copias de memoria innecesarias. Al modelar el tablero como una lista enlazada de filas, las filas de arriba no se tienen que mover en la memoria: simplemente se desconectan los punteros verticales (arriba y abajo) de la fila que se completó, se reconecta la fila inmediatamente superior con la inferior y se libera la fila eliminada. Las filas superiores bajan automáticamente porque sus enlaces ahora apuntan a la fila correcta sin tocar su contenido interno.

- Costo de eliminar una fila completa: Se recorren las 10 columnas desconectando los punteros verticales y liberando las celdas con delete. Esto toma exactamente 10 liberaciones y 20 reasignaciones de puntero, representando un costo de $O(C)$ operaciones (que es $O(1)$ tiempo constante porque el ancho de 10 columnas es fijo).
- Costo de insertar una fila vacía: Se crean 10 celdas vacías con new, se enlazan de izquierda a derecha y se conectan sus punteros abajo a la fila que estaba arriba, actualizando el puntero origen del tablero. Toma 10 creaciones y unas 30 asignaciones de puntero: costo $O(C)$ ($O(1)$ constante).

Pregunta 3: Necesidad de la lista doblemente enlazada para el replay y costo de avanzar y retroceder

Para implementar las funciones de deshacer y el replay necesitamos movernos en el tiempo en ambas direcciones (hacia adelante y hacia atrás). En una lista simplemente enlazada cada nodo solo conoce al nodo siguiente. Avanzar en el tiempo es directo (actual = actual->siguiente), pero para retroceder un solo paso es imposible regresar desde el nodo actual: habría que empezar a buscar desde la cabeza de la lista recorriendo nodo por nodo hasta encontrar cuál apuntaba al actual. Si la partida lleva k jugadas, cada pulsación para retroceder tomaría $O(k)$ operaciones, lo que causaría congelamientos y tirones de pantalla. La lista doblemente enlazada resuelve esto porque cada nodo guarda tanto su puntero siguiente como su puntero anterior.

- Costo de avanzar un paso (historial.avanzar() / Tecla Y o botón Replay Adelante): Solo hace actual = actual->siguiente. Toma $O(1)$ operaciones elementales.

- Costo de retroceder un paso (`historial.retroceder()` / Tecla Z o botón Replay Atrás): Solo hace $\text{actual} = \text{actual} \rightarrow \text{anterior}$. Toma $O(1)$ operaciones elementales.

Pregunta 4: Garantía del evento más próximo al frente y costo de inserción en la cola de eventos

Para la cola de eventos hicimos una estructura que se mantiene ordenada por tiempo. Cada vez que metemos un evento nuevo, se inserta y se va comparando con los elementos anteriores para acomodarse en su posición correcta según el segundo exacto en el que tiene que activarse. De esta forma, el evento que tiene que ocurrir más pronto siempre queda de primero (en la posición 0). Para revisar si ya le toca activarse, el juego solo tiene que consultar ese primer elemento en $O(1)$. Insertar un evento nuevo solo toma $O(\log n)$ operaciones porque se va dividiendo a la mitad en cada paso, lo cual es muy rápido y evita tener que ordenar toda la lista desde cero cada vez.